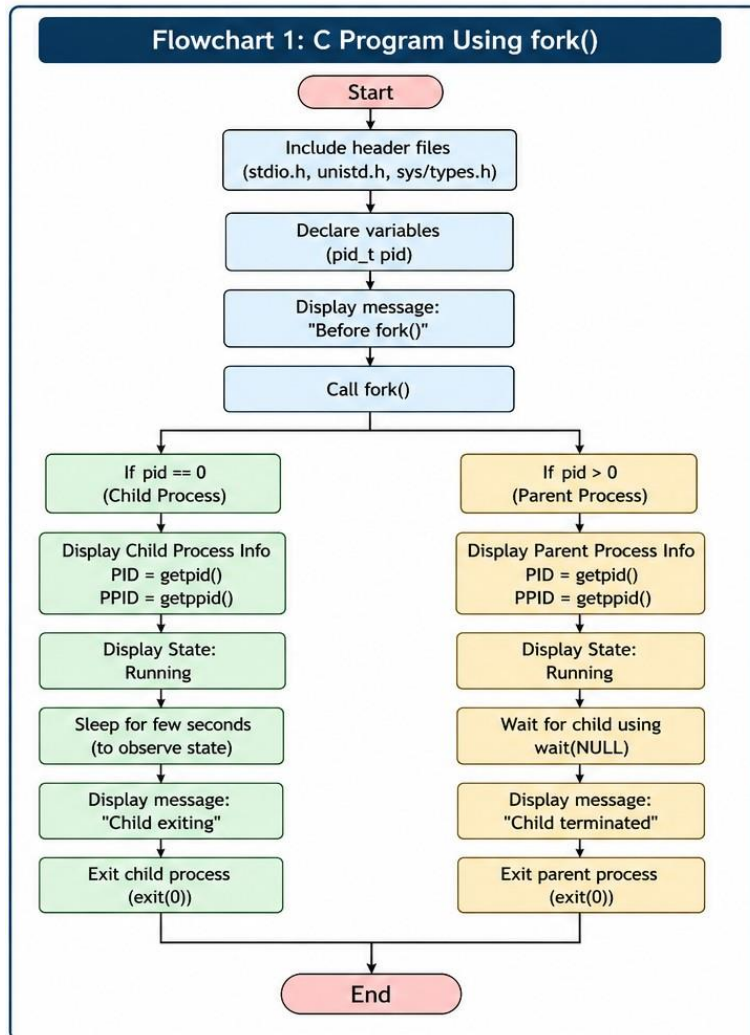


Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execuCon. Flow Chart



```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h> #include
```

```
<sys/wait.h>
```

```
int main() {
```

```

pid_t pid;

prinN("Before fork():\n");   prinN("Process PID = %d, PPID
= %d\n", getpid(), getppid());

pid = fork();

if (pid < 0) {    // Fork failed
fprinN(stderr, "Fork failed\n");
exit(1);
}

else if (pid == 0) {    // Child process    prinN("\n[CHILD]
Process created.\n");    prinN("[CHILD] PID = %d, PPID =
%d\n", getpid(), getppid());    prinN("[CHILD] State:
Running\n");    sleep(2); // Simulate work
prinN("[CHILD] TerminaCng.\n");
    exit(0);
}

else {
    // Parent process    prinN("\n[PARENT] Child created with
PID = %d\n", pid);    prinN("[PARENT] PID = %d, PPID = %d\n",
getpid(), getppid());    prinN("[PARENT] State: WaiCng for child
to terminate\n");

    int status;    waitpid(pid, &status, 0); // Parent waits
(WaiCng state)

```

```

        printf("[PARENT] Child (PID = %d) has terminated.\n", pid);
    printf("[PARENT] State: Running -> Terminated\n");
}

return 0;
}

```

ExecuCon:

```

Parent Process Started
Parent PID: 21749
Child 1 started. PID = 21753, Parent PID = 21749
Child 2 started. PID = 21754, Parent PID = 21749
All 3 child processes have been created.

Using wait():
Child 3 started. PID = 21755, Parent PID = 21749
Child completed. PID = 21753
wait() detected child PID 21753 finished with exit status 10

Using waitpid():
waitpid() detected child PID 21754 finished with exit status 11
waitpid() detected child PID 21755 finished with exit status 12

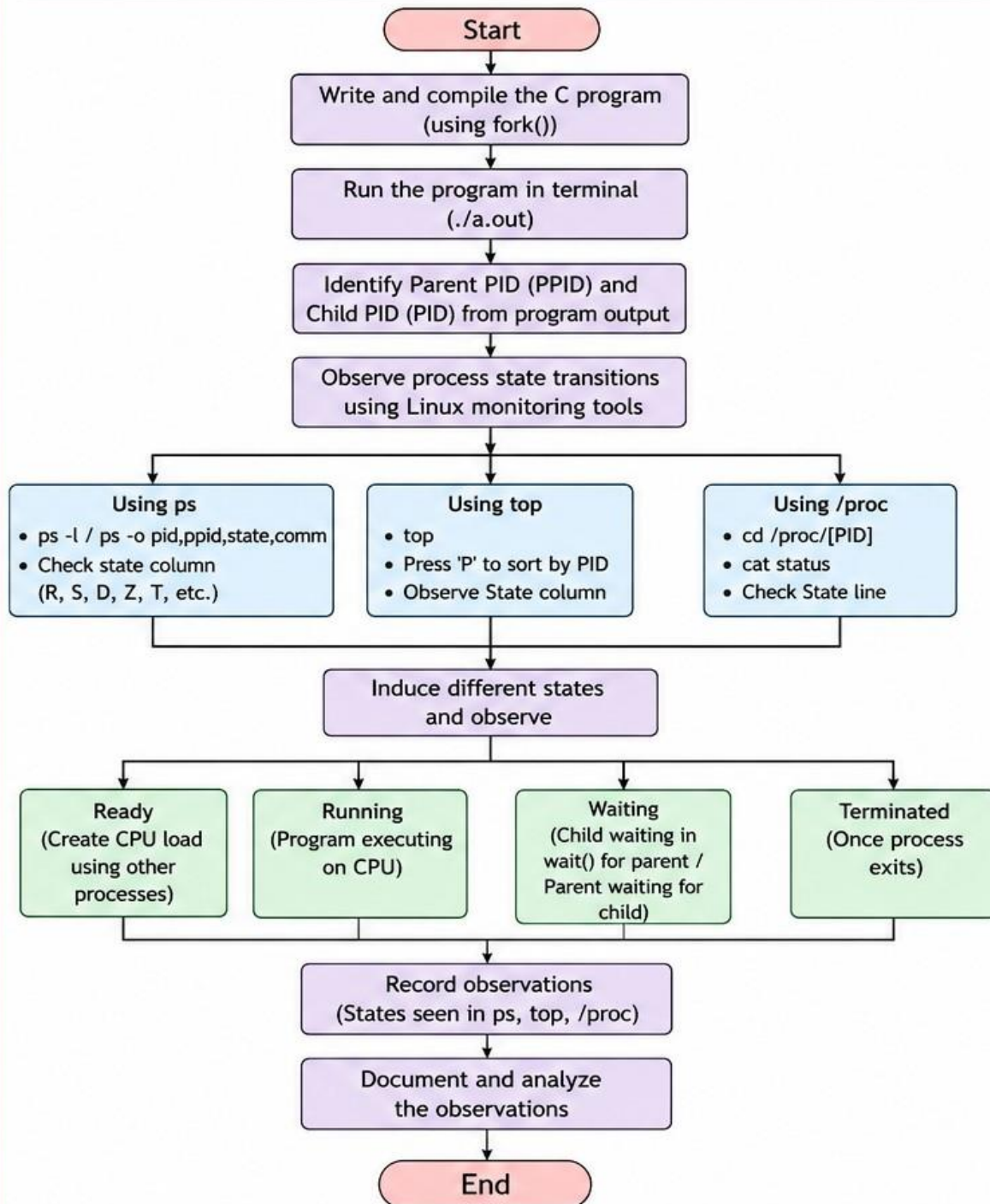
Parent process completed.

```

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Flow Chart:

Flowchart 2: Experiment to Observe Process State Transitions



#include <stdio.h>

```
#include <stdlib.h> #include
```

```
<unistd.h>
```

```
int main() {  prinN("PID = %d : Running (CPU-bound work)\n",
```

```
getpid());  for (long i = 0; i < 2000000000; i++);  // busy-loop -
```

```
> Running
```

```
    prinN("PID = %d : Going to sleep -> WaiCng/Blocked state\n", getpid());
```

```
    sleep(15);  // sleep() -> WaiCng (Interruptible sleep, state 'S')
```

```
    prinN("PID = %d : Woke up, finishing -> Terminated\n", getpid());
```

```
    return 0;
```

```
}
```

ExecuCon:

```
chakrika@Chakrikas-MacBook-Air 0SSP % ./ws-t4
PID = 22054 : Running (CPU-bound work)
PID = 22054 : Going to sleep -> Waiting/Blocked state
PID = 22054 : Woke up, finishing -> Terminated
chakrika@Chakrikas-MacBook-Air 0SSP %
```